

DriveGuardAI
Machine Learning-Based Intrusion Detection in Vehicular
Networks

Prepared By:
Mohammad Ibrahim Malkawi

Project Submitted in Partial Fulfillment of the Requirements for
the Degree of Science in Cybersecurity

Abstract

The rising level of connectivity among modern vehicles has significantly increased the level of cybersecurity threats targeting the communication capabilities used among vehicles, with the Controller Area Network (CAN) being the most impacted communication protocol. To mitigate the above challenges, the proposed project will utilize the CIC IoV Dataset 2024, which is a realistic dataset generated based on the communication protocol used among vehicles, namely the CAN protocol, which was collected from a real 2019 Ford vehicle operating under both benign and attacking conditions, including Denial-of-Service (DoS) and spoofing attacks. Under the proposed project, the DriveGuardAI system will be implemented as a machine learning-based intrusion detection system targeting the Internet of Vehicles (IoV) environment, the project will focus on data preprocessing, feature extraction, class imbalance handling, model development, model evaluation, among other steps, with the main objective being the development of multiple machine learning-based models that can efficiently detect any potential threats targeting the CAN protocol used among vehicles. Based on the proposed project, the following are the potential implications that can be derived: The proposed project will significantly contribute to the development of efficient intrusion detection mechanisms targeting the Internet of Vehicles (IoV) environment.

Keywords:

Internet of Vehicles (IoV), Intrusion Detection System (IDS), Denial-of-Service (Dos), Spoofing, Internet of things (IoT).

Table of Contents

Declaration of Originality.....	i
Acknowledgments	ii
Abstract	iii
Table of Contents	iv
List of Figures.....	vi
List of Tables.....	vii
Abbreviations	viii
Chapter 1 Introduction	1
1.1 Overview.....	1
1.2 Problem Statement.....	2
1.3 Project objectives.....	2
1.4 Significance of the project.....	3
1.5 Project Contributions.....	3
1.6 Outline of the report.....	4
Chapter 2 Literature Review	6
2.1 Related Work.....	6
2.2 Knowledge Gap and Motivation.....	8
Chapter 3 Project Planning	9
3.1 Deliverables.....	9
3.2 Project Tasks & Timeline.....	10
3.3 Roles and Responsibilities.....	11
3.4 Risk Assessment.....	11
3.5 Cost Estimation.....	13
3.6 Project Management Tools.....	14
Chapter 4 System Analysis & Requirements	14
4.1 System Overview.....	14
4.2 Functional Requirements.....	15
4.3 Non-Functional Requirements.....	16
4.4 Platform Requirements (Software/Hardware).....	16
4.5 Stakeholders.....	19
Chapter 5 System Design	20
5.1 System Architecture (diagram + description).....	20
5.2 Database or Data Model Design (ERD / schema).....	22
5.3 User Interface Design (screens or flow diagram)	24

Chapter 6 System Implementation.....	25
6.1 Development Tools and Technologies.....	25
6.2 Implementation Overview.....	26
6.3 Algorithms or Models Used.....	27
Chapter 7 Testing and Evaluation	32
7.1 Testing Approach (method, dataset, metrics).....	32
7.2 Results and Analysis.....	32
7.3 Discussion.....	36
Chapter 8 Conclusions and Future Work	37
8.1 Summary of Achievements.....	37
8.2 Limitations & Future Enhancements.....	37
References	39
Appendices	41
Appendix A User Manual.....	41
Appendix B Code Documentation or GitHub Link	41
Appendix C Supplementary Figures or Tables	41

List of Figures

Figure 1. DoS attack scenarios on in-vehicle network.....	1
Figure 2. Project Deliverables	9
Figure 3. Gantt chart for Timeline and tasks.....	10
Figure 4. diagram for system architecture	20
Figure 5. ERD for data model design	22
Figure 6. User interface design.....	24
Figure 7. overview page	24
Figure 8. Multi-Format Comparison	33
Figure 9. performance comparison.....	33
Figure10 . Confusion Matrix	34
Figure 11. ROC Curves	34
Figure 12. 5-Fold CV	34
Figure 13. 3-Case Comparison page	35
Figure 14. Feature Importance using SHAP	36

List of Tables

Table 1. Abbreviations.....	viii
Table 2. Risk Assessment	12
Table 3. Functional Requirements	15
Table 4. Non-Functional Requirements	16
Table 5. Software Requirements	17
Table 6. Hardware Requirements	18
Table 7. Stakeholders.....	19
Table 8 Effect of CAN-frame de-duplication.....	30
Table 9. four models comparison.....	35

Abbreviations

Abbreviation	Full Form
IOV	Internet of Vehicles
CAN	Controller Area Network
ECU	Electronic Control Unit
DOS	Denial of Service
ML	Machine Learning
DNN	Deep Neural Network
LR	Logistic Regression
RF	Random Forest
AB	AdaBoost
IDS	Intrusion Detection System
CSV	Comma-Separated Values
AI	Artificial Intelligence
SW	Steering Wheel
RBM	Revolutions Per Minute
CICIoV2024	Canadian Institute for Cybersecurity Internet of Vehicles Dataset 2024

Table 1. Abbreviations

Chapter 1 Introduction

1.1 Overview

On account of the strong development of smart and connected vehicles, the Internet of Vehicles IoV has become a major part of the modern transportation systems.

With IoV capabilities in enabling the vehicles to communicate with each other in road infrastructure and with cloud services it improve the safety efficiency and user experience. However in this interconnectivity it also increase the risk of cyberattacks that could exploit the vehicle's internal systems and putting the passengers at risk[1]. Now because we need to secure our system we need to talk about one of the most critical communication systems components inside the vehicle is the controller area network (can bus) which connects various electronic control units (ECU) that is responsible for functions such as braking, steering and engine control. With all of these advantages that we mentioned the (can bus) protocol has an important weakness in built in security mechanisms such as encryption and authentication making it vulnerable to attacks like spoofing and denial of service (Dos). In this project with the help of artificial intelligence and machine learning techniques we are going to detect the attacks in IoV environments by using CICIoV2024 dataset [2] , which contains real world can bus traffic and labeled attack samples the project aims to train and evaluate intelligent model that is capable of identifying abnormal or malicious network behavior in connected vehicles.

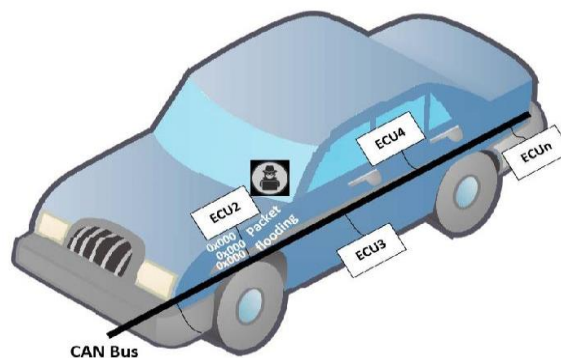


Figure 1. DoS attack scenarios on in-vehicle network.

1.2 Problem Statement

With the rise in connectivity in vehicles, traditional in-vehicle network communications like the CAN bus are becoming vulnerable to various forms of cyber-attacks. This is because of the lack of security capabilities within the protocol, which allows attackers to introduce malicious messages, impersonate genuine ECU, and send high-priority data on the network to cause malfunctions and thus pose a threat to safety, leading to the risk of losing control over the vehicle. The current methods for detecting such attacks involve intrusion detection systems (IDS), which are mostly based on predefined rules or patterns and thus ineffective in detecting novel and rare attack forms. Failure to resolve this problem would mean that autonomous vehicles remain at constant risk of attacks, posing a major safety concern and reducing trust in self-driving vehicles. In our project we are going to focus on developing an AI based intrusion detection system for IoV networks using the CICIoV2024 dataset our goal is to enhance the ability to detect and classify can bus attacks accurately and efficiently leading to safer and more secure intelligent transportation systems.

1.3 Project objectives

Our achieving goals of this project are to develop analyse and evaluate the best machine learning based models using the CICIoV2024 dataset and improving cybersecurity in Internet of Vehicles by:

1. Developing machine learning based Intrusion Detection System (IDS) that has the ability of detecting DoS and spoofing attacks in IoV can bus traffic. And evaluating the performance of different ML algorithms like Logistic Regression, Random Forest, AdaBoost, and Deep Neural Network in classifying benign and malicious traffic.
2. Developing and evaluating different exciprailable AI models (eg shap , lime) to guarantee the transparency and accountability of the DriveGuardAI system. The project uses a combination of both SHAP and LIME to offer a logical reason why a particular message on the CAN bus has been labeled as a DoS or a Spoofing attack. This makes the complicated algorithms in the machine learning system less of a "black box" and, in effect, lowers the number of false positives in the system, promoting the safety of IoV environments.
3. providing recommendations for the future of iov security mechanisms

1.4 Significance of the project

We choose this project because it addresses one of the most critical challenges in modern transportation systems by securing the Internet of Vehicles (IoV) against modern cyberattacks because of the increasing of interconnection between vehicles, road infrastructure and smart city systems the potential impact of cyber threats like Denial of Service (DoS) and spoofing attacks has become major concern in the world and can be done by utilizing the CICIoV2024 dataset we could contribute to the cybersecurity practice and research by providing a detailed analysis of real vehicular communication data that collected from 2019 ford platform. Our project improves the understanding of how Machine Learning (ML) models can detect and classify malicious traffic within the can bus. the outcomes of this project can help cybersecurity professionals improve the system safety and reduce the risks of cyberattacks on connected cars. and with improving IoV security it should directly increase the safety and reliability of intelligent transportation systems improving both drivers and the public by preventing accidents and ensuring secure mobility in smart cities.

1.5 Project Contributions

In our project we will build a machine learning based intrusion detection model to enhance the security of the Internet of Vehicles (IoV) by using the CICIoV2024 dataset we will focus on our work to detect and prevent attacks that target the vehicles can bus communication system by these steps:

- We will design a new detection framework that can accurately identify spoofing and Denial of Service (DoS) attacks within IoV networks.
- We will apply advanced data preprocessing and feature extraction techniques to improve model accuracy and reduce redundant CAN traffic.
- Several machine learning algorithms will be tested and compared to find the most effective approach for real-time intrusion detection.
- We will carry out a detailed performance evaluation using standard metrics such as accuracy, precision, recall, and F1-score.
- And finally, we will propose an improved IoV security framework that can be adapted for future connected and autonomous vehicle systems.

1.6 Outline of the report

Chapter 1 introduces the concept of cybersecurity on the Internet of Vehicles (IoV) and highlights the growing risks of attacks on the CAN bus system. It explains the motivation behind developing a machine learning-based intrusion detection model using the CICIoV2024 dataset. The chapter also presents the problem statement, project objectives, significance, and main contributions of the project.

Chapter 2 reviews the existing literature related to intrusion detection in vehicular and IoV networks, summarizing prior approaches based on simulation data, malware datasets, and various machine learning and deep learning techniques. It then identifies the knowledge gaps in current research, such as reliance on simulators and limited consideration of data representation, which motivate the direction of this project.

Chapter 3 presents the planning of the project, outlining its deliverables and phases of work, the project timeline and tasks, the roles and responsibilities of each team member, a risk assessment of potential threats to the project's assets, the estimated cost of the project, and the tools used for project management and collaboration.

Chapter 4 covers the system analysis and requirements of DriveGuardAI, including an overview of the proposed system, the functional and non-functional requirements it must satisfy, the platform requirements in terms of software and hardware, and the stakeholders affected by or interested in the system.

Chapter 5 describes the design of the DriveGuardAI system, including its multi-layered system architecture, the entity-relationship data model used to organize CAN traffic, detection logs, and model performance data, and the design of the user interface through which analysts interact with the system.

Chapter 6 details the implementation of the DriveGuardAI system, including the development tools and technologies used, the end-to-end pipeline from data ingestion through model training and deployment, the machine learning and deep learning algorithms implemented, and insights from recent CICIoV2024 research that informed key design decisions, particularly regarding dataset redundancy and model selection.

Chapter 7 presents the testing and evaluation of the system, describing the testing methodology, dataset split, and evaluation metrics used, followed by the results and analysis of the four classifiers across binary and multi-class classification tasks, multiple data representations, and the three-case de-duplication comparison, along with a discussion of the findings.

Chapter 8 concludes the report by summarizing the achievements of the project relative to its original objectives, and by discussing the limitations of the current system along with proposed future enhancements such as sequence-based models, anomaly detection, cross-vehicle generalization, and edge deployment.

Chapter 2 Literature Review

2.1 Related Work

The authors in [3]: Dataset: Synthetic traffic simulation data

The authors proposed that spoofing attacks in CAVs are simulated in this work using OMNeT++, SUMO, and Veins for an analysis with respect to vehicle behaviour and network performance. Metrics used for this purpose include PDR, speed, acceleration, and CoV for driving instability. Results depict that there is an increased speed fluctuation, lower PDR, and higher collision risk during spoofing. This study is realistic but limited because it examines only spoofing without providing an ML-based IDS [3].

The authors in [4]: Dataset: Windows Executables Dataset (Virus Share): 1000 malware samples and 1000 benign apps for proof-of-concept. <https://virusshare.com/> This paper proposes a hybrid static-dynamic malware detection framework based on a dataset of Windows malware and several ML models: RF, J84, XGBoost, and NB. Feature selection is done by using InfoGain to improve the accuracy. vRF and XGBoost had a strong performance of ROC-AUC ≈ 0.98 , but the dataset is not CAN-Bus or vehicle-specific and dynamic analysis is too heavy for real-time automotive deployment [4].

The authors in [5]: The presented study compares ML models, namely SVM, DT, RF, and KNN, for CAN-Bus attack detection using simulated traffic derived from real vehicle signals. Random Forest showed the highest accuracy, at about 99%. The limitation is the use of simulated data and lack of real-vehicle testing, plus no deep-learning methods [5].

The authors in [6]: The paper proposes a CNN model utilizing multi-scale histograms of CAN messages in order to learn spatial-temporal attack patterns including DoS, spoofing, and fuzzing. Even though the CNN achieved high accuracy at 98.7%, it detected unseen attacks, but it is computationally intensive and may not run efficiently on embedded vehicle hardware [6].

The authors in [7]: It presents a hybrid model of the IDS which combines XGBoost for feature selection and the DNN for classifying, based on real CAN-Bus traffic from an autonomous vehicle. It succeeds with 99.9% accuracy in the hybrid model but requires considerable computation and uses a very limited dataset without varied attack scenarios [7].

The authors in [8]: It focuses on improving IDS accuracy on imbalanced IoV datasets using oversampling/undersampling and ML models like KNN, NB, and LDA. The dataset balancing yielded significant improvements, with KNN doing best. However, the dataset is small and lacks deep-learning evaluation, neither does it leverage modern datasets such as CICIoV2024 [8].

The authors in [9]: The authors have proposed an Intrusion Prevention System in IoV based on deep learning with DNN, CNN, and RNN models. It was trained on two major intrusion datasets, namely KDD99 and CICIDS 2018, the scheme consists of three stages: preprocessing, feature extraction, and classification. Among all models, the highest performance was achieved using DNN with 99.57% accuracy, which made it the most effective for IoV attack detection. Although the system benefits from a high accuracy and strong datasets, its limitation arises because the proposed system relies on offline data without real-time IoV testing. The authors therefore suggest that, in the future, real-world IoV datasets shall be used in order to deploy this model in the field for its performance evaluation in dynamic network conditions [9].

The authors in [10]: In this paper, an IoV Intrusion Detection System with an improved random forest classifier is proposed, which leverages the power of Extra Trees and SMOTEBoost. Based on the CIC-IDS2018 dataset, high accuracy of about 99% is achieved with low latency, especially in the detection of DoS attacks. The system reduces false positives and improves detection performance; however, it is still struggling with rare attack types such as U2R and Botnet. The authors recommend continuing this work for the detection of such attack types and gathering more extensive and bigger datasets in real-time for more realistic evaluation [10].

Conclusion Related work

The reviewed research demonstrates that deep learning and machine learning models like ensemble methods and deep learning models (Random Forest, XGBoost, etc.) can achieve very high accuracy when detecting spoofing or DoS attacks on CAN-bus and IoV networks. That said, there are clear limitations of published studies in this area such as reliance on simulations or out-of-date datasets not specific to vehicles for input data, much higher computational costs than would be practical for any type of in-vehicle computer hardware, investigations concentrate primarily on one specific type of attack, and lack of consideration of how CAN data representations affect detection abilities. Because of these shortcomings, our work focuses on utilizing the newly provided CICIoV2024 dataset collected in everyday-use environments to develop a lightweight, efficient, explainable IDS that can detect multiple types of attacks and examine various representations of data sets.

2.2 Knowledge Gap and Motivation

Although many studies have addressed security issues in in-vehicle communication systems, there are many research gaps exist and still remain, need to be fulfilled. First, one of the major research gaps in previous studies is their dependence on CAN-Bus data simulators. As previously explained, simulators are useful in analyzing the attacks, but they are unable to simulate the delays, timing variations, noises, and irregular patterns that appear in real vehicles which are common in real vehicles. Moreover, many researchers focus on one attack approach at a time, neglecting the fact that many attacks can occur at the same time in modern connected vehicles. The next gap is in the representation and processing of data in the CAN-Bus system. Many researchers limit data representation to only one format, either in decimal or in hexadecimal, without considering whether representing the data in another format can help in increasing detection accuracy. Also, some developed IDS systems for vehicle security use deep learning methods, which require high performance computing systems like strong hardware, which in many cases, specifically in vehicles, are not available. These gaps indicate a clear need for a more realistic and practical solution. The idea that drives the development of this project is based on a recent set of data that is gathered from a fully operational car (CICIoV2024), because there is CAN-Bus behaviour and attack scenarios associated with it. The development aims to explore different machine-learning techniques using different data forms for reliable and lightweight detection.

Chapter 3 Project Planning

3.1 Deliverables

The project is divided into phases:



Figure2. Project Deliverables

The figure above summarizes the six sequential steps of the completed project, with each step based on the deliverables created in the prior phase. The first phase of the project was to understand the problem and needs, where the main issues associated with the security of the IoV were identified and main attacks on autonomous vehicles connected via a CAN bus were evaluated. The second phase of the project was to complete a literature review, where prior research in AI-based intrusion detection for IoV platforms was located and reviewed to provide context for the study. The third phase was to develop a dataset for creating and validating a machine learning-based intrusion detection system, this involved

analyzing and extracting information from the CICIoV2024 dataset and developing preprocess strategies that included class-balancing procedures to prepare the dataset for training. Phase four was to develop and train machine and deep learning-based intrusion detection models to identify anomalies on the CAN bus. Phase five was to evaluate and document the models by evaluating use of standard performance metrics to test the validity of the models and to create a clear audit trail for documenting steps and results for users to duplicate the study.

The Final Presentation stage of the project is the culmination of all results, putting together the final report, slide presentation, and a live demo of the intrusion detection system. All of these stages form either an entire 'workflow' for the project, leading to a full problem statement, solution, and proof of working of the solution.

3.2 Project Tasks & Timeline

We will implement this project into five main phases: analysis, design, implementation, testing, and documentation.

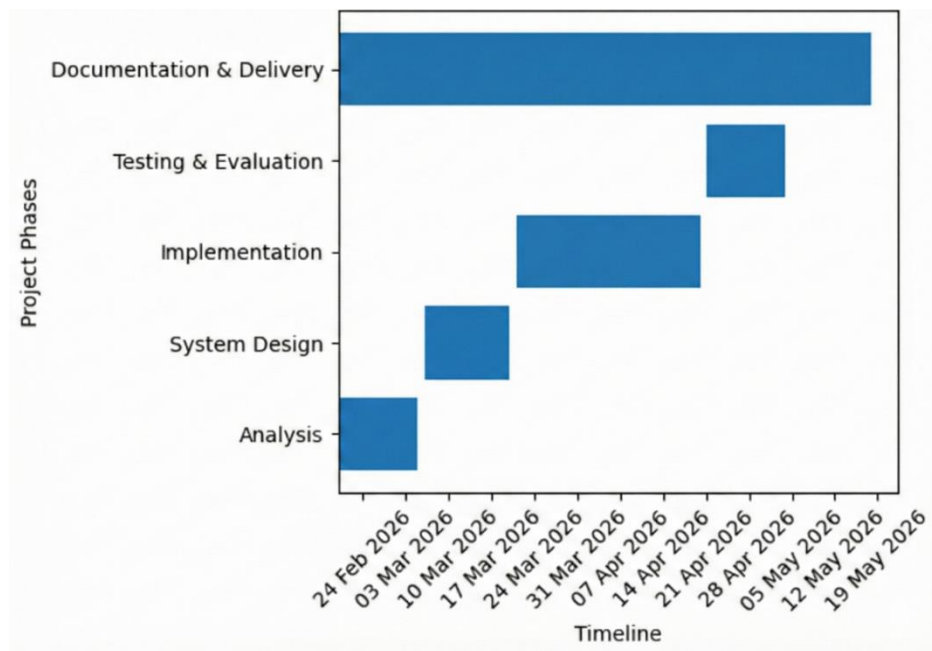


Figure 3. Gantt chart for Timeline and tasks

3.3 Roles and Responsibilities

- **Mohammad Ibrahim Malkawi 164473**: Select the machine learning models to be used for the IDS and preparing the comparison criteria for different algorithms that we will Implement, And assisting the team in other shared tasks.
- **Mousa Mohammad Alshaer 163770**: Conduct the literature review, analyze the previous IDS research, and prepare the evaluation plan including metrics such as accuracy, precision, recall, and F1-score And assisting the team in other shared tasks.
- **Abedalaziz Zeyad Alweher (160317)**: Preparing the functional and non-functional requirements of the system, designing the system architecture, listing the stakeholders, And assisting the team in other shared tasks.
- **Ameer Ahmad Alfaqeeh (161194)**: Understanding the dataset of CICIoV2024, planning the steps for data preprocessing (cleaning, formatting, and balancing) and structuring or organizing the project report, And assisting the team in other shared tasks.

3.4 Risk Assessment

Prior to execution, key risks associated with the project have been identified and assessed to ensure asset protection and avoid any potential harm to data and the results of the project. Table 2 outlines a structured risk assessment approach, where each risk is tied to specific assets, ranked in terms of their likelihood and impact, resulting in risk level classification, and addressed through a mitigation strategy.

Assuming we do the logical and physical requirements to provide security.

#	Risk	Assets	Likelihood	Impact	Risk Level (I*L)	Mitigation
1	Loss or corruption of data, processed files, or original source code.	Raw data set (hexadecimal, decimal, binary), Processed data	Low: Let's assume we have multiple backups distributed across more than one device and are aware of this risk.	High: Work may have to be repeated.	Medium	Use version control, keep regular backups , and document all, Git + signed commits, offline backups , verify hashes, document preprocessing steps.
2	Unauthorized access to dataset/notebooks.	Storage, notebooks, credentials.	Low: Let's assume we want to use a strong and secure authentication and login method.	High	Medium	Least privilege, MFA, encrypt storage, avoid public links, rotate secrets, Session key
3	Poisoned data or wrong labels during preprocessing.	Labels, Trained models	Low: Assuming read-only originals + automated validation checks + logging/versioning are in use	High	Medium	Read only originals, schema + class checks, pipeline logs, integrity validation.
4	Leak of attack traces or model artifacts.	Reports, features, Model files.	Low: Assuming strict access control, private storage, and only aggregate/non-sensitive data is shared)	Medium	Low	Only share aggregates, not artifacts, keep artifacts private, remove public repositories, access control.

5	Untrusted download/source (tampered file or malware).	Workstations, Python env, or dataset archives.	Low: Assuming only official sources of UNB/CIC are used, files are processed, hashes are validated, and analysis takes place in a isolated network environment.	High	Medium	Use official CIC/UNB data; scan files, check hashes, confine analysis to isolated environments like VMs
6	Adversarial CAN frames evade IDS (false negatives).	IDS Model, Thresholds, Deployment of IDS.	Low: One assumes that robust adversarial testing, threshold tuning/ensemble methods, monitoring, and periodic retraining are in place.	High	Medium	Robust testing, adversarial testing, ensemble/thresholds, monitor drift, retrain.
7	Theft/model inversion if model/API is exposed.	Model artifacts, APIs, logs.	Low (Because the model and artifacts are kept private, access is restricted to authorized users only, and the system is not publicly exposed)	Medium	Low	Restrict access; encrypt models; auth + rate-limit; monitor usage.

Table 2. Risk Assessment

As can be seen from the analysis, most of the risks can be classified as low after implementing the above-listed controls (versioning, offline storage, limited access, hash checking). The most notable risks were data corruption and wrong labels during preprocessing.

3.5 Cost Estimation

Developing the IoV Intrusion Detection System does not require a large budget, and most tools we use are free. The main cost comes from hardware if we choose to test the system on an actual vehicle network. A basic CAN bus interface or an OBD-II reader is usually enough for testing, and these devices typically cost between 50 and 120 dollars. For software, we do not need to purchase anything. The cost for using Python, machine learning tools, Jupyter Notebooks, VS Code, and any corresponding tools will be free since they have open-source licenses. We will incur expenses on printing official

documents or managing project paperworks, estimated at 10 to 20 dollars. Cloud computing will incur an optional fee of 5 to 15 dollars if we choose to train our model on fast hardware. The total cost of the project is expected to be anywhere from 60 to 150 dollars, depending on whether new hardware components are also being acquired.

3.6 Project Management Tools

To organize the development process and keep all team members aligned on their tasks, the project was divided into clear phases, with each phase building on the deliverables of the one before it, as described in Chapter 3. Google Colab served as the main development environment, allowing the team to write, run, and share the project's Python notebooks while gaining free access to GPU resources for training the Deep Neural Network, without requiring any local setup. Google Drive was used to store the CICIoV2024 dataset in its decimal, hexadecimal, and binary formats, the trained model and scaler files saved with joblib, and shared drafts of the report, diagrams, and screenshots, ensuring all members had access to the same resources and that previous versions of files remained backed up. For the system architecture and ERD diagrams presented in Chapter 5, Draw.io was used to produce clear, professional diagrams that could be easily updated as the design evolved.

Chapter 4 System Analysis & Requirements

4.1 System Overview

This section describes the functional requirements of the proposed system DriveGuardAI, being developed as an IDS based on ML for IoV applications. They describe the necessary functions that should be executed by this system to mitigate cyber attacks on vehicular CAN bus communications. The system should load and read the CICIoV2024 dataset and be able to detect the relevant attributes of the CAN bus traffic. The system should also do data preprocessing on the dataset and take care of missing values, eliminate unwanted data entries, perform encoding on the attributes that are categorical, and finally normalize the numeric attributes. Consequently, in order to effectively address the issue associated with the unbalances in the number of normal versus attack instances, proper methods aiming at balancing the data should be used. Additionally, the system should have the ability to pick the best significant features that result in a high level of accuracy for the detection process of intrusions. This directly contributes to the effective improvement associated with performance as well as the reduction in complexity. The system will be trained on various machine learning algorithms using the dataset that is learned in normal

as well as malicious communications in vehicular communications. It should detect and classify CAN bus traffic as either normal or malicious, including common attack types like spoofing and Denial of Service (DoS) attacks. The system will use standard evaluation metrics like accuracy, precision, recall, and F1-score to measure the performance of the trained models. Comparing these different models using such standard evaluation metrics will help the system to highlight which one of them provides the best results. Lastly, visualize the outcomes of the results through a confusion matrix, save the picked model for later usage, or reutilize it in further development.

4.2 Functional Requirements

Functional requirements encompass the actions that the DriveGuardAI system should be capable of executing in order to achieve its intended functionality. These requirements are listed in **Table 3** along with an individual identifier number and a categorization whether this action is essential or optional for the functioning of the system.

ID	Requirement	Type
1	The system shall monitor can bus network traffic to detect abnormal behavior	Essential
2	The system shall identify and classify Iov cyber-attack types using machine learning models	Essential
3	The system shall preprocess raw can bus data before training and testing	Essential
4	The system shall apply data balancing techniques to handle class imbalance	Essential
5	The system shall train and evaluate multiple machine learning models	Essential
6	The system shall visualize detection and classification results	Optional
7	The system shall generate evaluation metrics for performance comparison	Optional

Table 3. Functional Requirements

It can be seen from the table that the primary value of the system will come through fulfilling the essential requirements such as monitoring the CAN traffic, attack classification, preprocessing, balancing and training. Visualization and metric generation are considered optional and provide extra functionalities.

4.3 Non-Functional Requirements

In addition to capabilities of the system, quality attributes will influence the system's actual performance in the real environment. Non-functional requirements for the DriveGuardAI system classified according to specific categories like performance, security, scalability, and usability are represented in **Table 4**.

ID	Requirement	Category
1	The system shall process at least 1000 can packets per second	Performance
2	The false positive rate shall not exceed 1%	Performance
3	Model inference time shall support near real time detection	Performance
4	Access to datasets and experimental results shall be restricted to authorized users	Security
5	The system shall support future extension with additional datasets and attack types	Scalability
6	The system interface shall be understandable to users with basic technical knowledge	Usability

Table 4. Non-Functional Requirements

Among other considerations, it is vital that detection happens not only quickly but also accurately: the system should be able to perform near real-time inference, have less than 1% false positives rate, and handle at least 1000 packets per second. This is due to the necessity to deploy IDS in the vehicle itself.

4.4 Platform Requirements (Software/Hardware)

4.4.1 Software Requirements

The implementation of the system is predicated on having a clear software environment for all three stages mentioned (data processing, models development, and evaluation). This is reflected by **Table 5**, where the list of all software components employed within the project is shown with the indication of bare minimum and preferred versions.

Component	Minimum	Recommended / Notes
Operating system	Windows 10/11 or Ubuntu 22.04 LTS	Linux is recommended for live CAN capture (SocketCAN + can-utils). Windows is sufficient for offline training/evaluation.
Python	Python 3.10+	Use isolated environments (venv/conda) for reproducible experiments, and set of libraries for machine learning and fast development.
Data processing	pandas, NumPy	Load CSV files, clean data, engineer features, and convert hex/dec/bin representations.
Machine learning	scikit-learn	Logistic Regression, Random Forest, AdaBoost, evaluation metrics, and pipelines.
Imbalanced learning	imbalanced-learn	Optional balancing techniques (e.g., SMOTE/undersampling) to improve minority attack detection.
Deep learning (for DNN)	TensorFlow/Keras or PyTorch	Needed only for the DNN model; CPU-only execution is acceptable for small-scale runs.
Visualization	matplotlib	Confusion matrix, ROC curves, and comparative plots for results reporting.
Experiment environment	Jupyter Notebook/JupyterLab	Exploration, preprocessing, and repeatable experiment notebooks.
Development & collaboration	VS Code / PyCharm, Git/GitHub	Code management, collaboration, and versioned experiment artifacts.
Live CAN capture (optional)	python-can; can-utils (Linux)	Interfaces with CAN adapters and captures frames; SocketCAN backend recommended for Linux.
Containerization (optional)	Docker	Enables consistent runs across different machines and simplifies dependency setup.

Table 5. Software Requirements

It can be seen that the stack used in the project is completely dependent on open-source Python library packages (pandas, scikit-learn, TensorFlow/Keras, and the rest), ensuring that the work remains both reproducible and cost-effective.

4.4.2 Hardware Requirements

In order not to rely on any special infrastructure, the system was developed for commonly available hardware. **Table 6** demonstrates hardware requirements for the project with respect to development and testing, highlighting the distinction between minimum and recommended configurations along with some optional hardware for capturing CAN messages.

Component	Minimum	Recommended / Notes
CPU	Intel/AMD 64-bit dual-core (or equivalent)	Quad-core+ recommended for faster preprocessing and training.
Memory (RAM)	8 GB	16 GB+ recommended when working with large CSV files and multiple experiments.
Storage	10 GB free	SSD recommended; allow extra space for multiple dataset representations and saved models.
GPU (optional)	Not required	NVIDIA GPU (CUDA-capable) recommended only to accelerate DNN training.
CAN interface (optional for live testing)	USB-to-CAN or OBD-II CAN adapter	Examples: CANable/USB-CAN devices; ensure driver support for your OS (SocketCAN on Linux or vendor drivers on Windows).
Network	Wi-Fi/Ethernet	Required for dataset download, collaboration (Git), and optional cloud compute usage.

Table 6. Hardware Requirements

As can be seen from the above table, the project does not require sophisticated computer equipment but can be implemented with commonly available PCs, where a GPU and CAN adapters are purely optional.

4.4.3 Optional Deployment Targets (Near Real-Time Inference)

If the trained model is later integrated into a prototype for in-vehicle or edge testing, one of the following targets can be used depending on latency and compute needs: Standard vehicle diagnostic laptop connected to a CAN adapter (recommended for experiments and demonstrations). Embedded edge computer (e.g., Raspberry Pi 4/5) for lightweight ML models (LR/RF/AB) running on CPU. Embedded GPU edge platform (e.g., NVIDIA Jetson family) when a DNN model is required with low inference latency.

4.5 Stakeholders

Stakeholders are individuals or organizations that use DriveGuardAI, benefit from its outputs, or influence its requirements and constraints.

Stakeholder	Goals / Interests	Interaction with DriveGuardAI
Project team (developers)	Design, implement, and evaluate the IDS models and preprocessing pipeline.	Develop notebooks/code, run experiments, and document results.
Project supervisor and examiners (JUST/FCIT)	Ensure academic quality, correctness, and alignment with capstone requirements.	Review deliverables, provide guidance, and assess outcomes.
Vehicle cybersecurity analysts / SOC teams	Detect and investigate anomalous CAN behavior and suspected attacks.	Use model outputs/metrics and confusion matrices to tune thresholds and respond to incidents.
Automotive OEMs and Tier-1 suppliers	Improve vehicle security, reduce risk of exploitation, and support secure-by-design development.	Adopt findings to enhance in-vehicle IDS, validate ECUs/firmware, and inform product security decisions.
Fleet operators / mobility service providers	Maintain service availability and safety for connected fleets (taxis, delivery, buses).	Integrate IDS alerts into fleet monitoring and maintenance workflows.
Researchers and students	Benchmark new IDS approaches on realistic IoV data and reproduce experiments.	Reuse the pipeline and evaluation methodology for further studies.
Drivers and passengers (end beneficiaries)	Safety, reliability, and trust in connected/autonomous vehicle functions.	Indirect beneficiaries through reduced risk of attack-induced malfunctions.
Regulators and standards bodies	Promote compliance and baseline security practices for connected vehicles.	Use insights to align with regulations and standards (e.g., cybersecurity management and risk assessment).
Penetration testers / red teams (authorized)	Validate security controls and measure resilience against spoofing/DoS techniques.	Test the system, report weaknesses, and propose mitigations.

Table 7.Stakeholders

The variety of stakeholders from the development team and academic mentors all the way down the line to the end-users, the OEMs, SOC analysts, governmental bodies, and finally, the drivers and passengers, makes it clear that the potential of the system does not end at the academic level.

Chapter 5 System Design

5.1 System Architecture (diagram + description)

The DriveGuardAI system architecture is designed in a multi-layered manner in a modular setup that is efficient in processing fast vehicular network traffic and identifying cyber-attacks. In **Figure 4**, the DriveGuardAI system architecture has four major layers: Data Source Layer, Preprocessing Layer, Intrusion Detection Engine, and Application and Storage Layer.

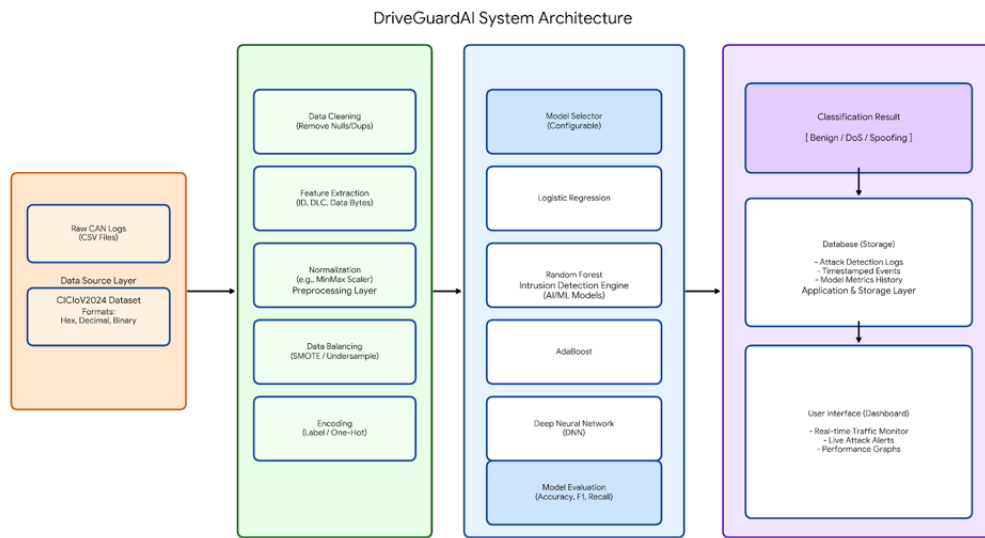


Figure 4. diagram for system architecture

1. **Data Source Layer:** The input for this system comes from the CICIoV2024 Dataset, which is a simulation of real-world CAN bus traffic for a 2019 Ford car. The system architecture is designed to process data that comes in three different types: Hexadecimal, Decimal, and Binary. This serves as a point of entry for simulating a never-ending stream of CAN messages (containing Arbitration IDs, DLCs, and Data) to the system.
2. **Preprocessing Layer:** The raw data obtained from the vehicular systems needs to be extensively prepared before the process of analysis. There are 4 major tasks in this layer.

3. **Data Cleaning:** This phase includes the elimination of duplicate data as well as null values. **Feature Extraction:** In this step, the most significant features are chosen. These are the features that carry the signature of the attack. These could include Arbitration ID and DATA_0 to DATA_7. **Normalization:** This involves scaling the numerical variables using methods like Min-Max Scaling in a manner that can be used with the Deep Neural Network (DNN).
4. **Data Balancing:** It includes the use of algorithms such as the Synthetic Minority Over-sampling Technique (SMOTE) or random undersampling to deal with the unbalance in the amount of benign data and the data relating to attacks such as DoS and Spoofing.
5. **Intrusion Detection Engine:** This component makes decisions. It runs different models based both on Machine Learning and Deep Learning paradigms to compare performances and enhance intrusion detection capabilities. The Machine Learning models that are proposed to be implemented in the project are the following: the Logistic Regression (LR) model, the Random Forest (RF) model, and the AdaBoost model. **Deep Learning:** A Deep Neural Network (DNN) is used to identify the non-linear patterns in the traffic data. **Model Evaluation:** The models are continuously evaluated based on standard evaluation metrics (Accuracy, F1-Score, Recall) to determine the best classifier for the prevailing conditions.
6. **Application and Storage Layer:** The last results are managed in this layer:
classification Result: The traffic is classified as “Benign” or as a particular attack (“DoS”, “Spoofing-RPM”, “Spoofing Database: The structured database records the logs of detections, the attack events marked by timestamps, and the history of model performance. **User Interface (Dashboard):** Real-time information is provided to the security analyst using a graphical interface, which may include traffic displays, attack notifications, and graphical statistics of performance.

5.2 Database or Data Model Design

Database Relationship Description

This section expands on the logical relationships between the entities that exist within the DriveGuardAI system, as a means of explaining how the data model addresses the goals of the project for vehicular network intrusion detection and classification.

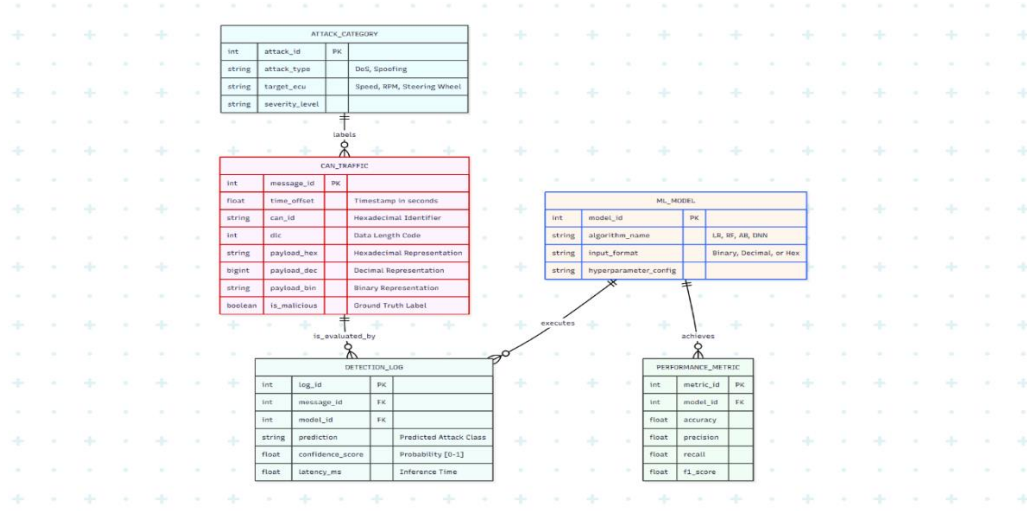


Figure 5. ERD for data model design

1. **ATTACK_CATEGORY to CAN_TRAFFIC (1:N):** There is a one-to-many relationship between the Attack_Category and CAN_Traffic tables. Every network packet that is captured from the CIC IoV 2024 dataset corresponds to a particular type of attack category, like Denial of Service (DoS) attacks, Spoofing attacks (like Speed, RPM, and Steering Wheel attacks), and others that belong to the category of Benign traffic attacks.
2. **CAN_TRAFFIC to DETECTION_LOG (1:N):** A single instance of the CAN traffic can be related to multiple detection records. Since the objective of the project is to compare the performance of different algorithms such as Logistic Regression, Random Forest, AdaBoost, and Deep Neural Networks, a single instance of the CAN traffic can be processed by different algorithms to identify which algorithm performs the best.
3. **ML_MODEL to DETECTION_LOG (1:N):** This relationship connects the machine learning models to the detection logs produced as a consequence of the models being executed. Each model produces several detection logs depending on the input provided to the model. One of the most important parameters in this relationship is the Latency (Inference Time), which is measured to ensure that the detection process takes place in near real time, as required in the vehicle environment.

4. ML_MODEL → PERFORMANCE_METRIC (1:N): Each model is associated with a list of performance metrics that are evaluated during the test phase. This association enables the storage and comparison of standardized test results, such as Accuracy, Precision, Recall, and F1-score, which helps in performing an in-depth comparison of the algorithms used.

5. Integration phase : Multi-Format Data Representations: The CAN_Traffic entity will be designed in such a way that it holds different attributes for Hexadecimal, Decimal, and Binary formats of the data payload. The structure of the CAN_Traffic entity itself enables one of the major objectives of the project, which is to assess the impact of the varying forms of the information results of the IDS.

5.3 User Interface Design

The user interface of the DriveGuardAI system will have two pages. The first page will be for the login purpose, and the second page will be for the receipt of CAN bus information, detection criteria, and execution of the intrusion detection models. The detection information, attack information, and other metrics of the models will be represented more visually on this website than it is now.

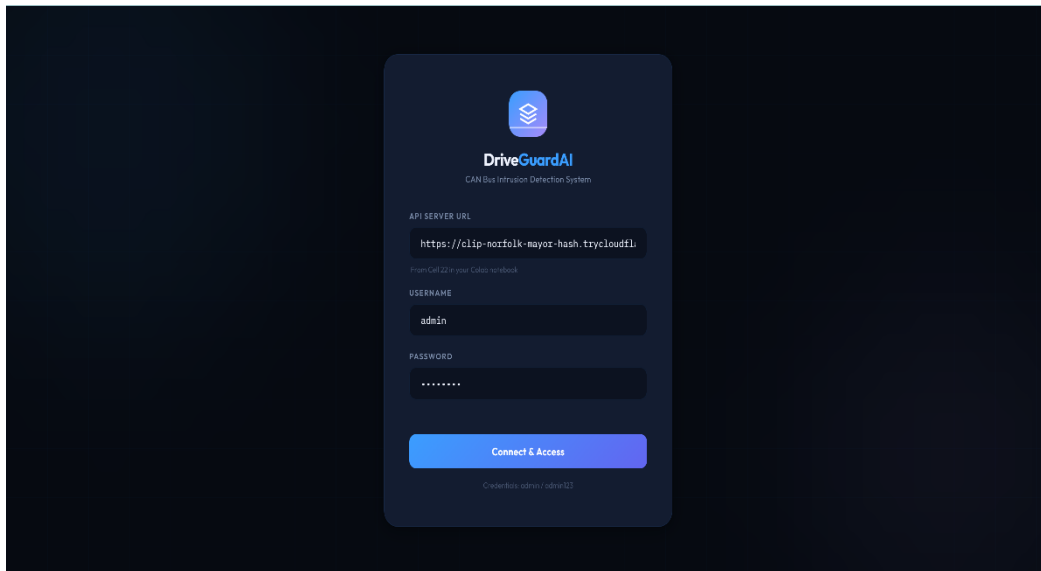


Figure 6. User interface design

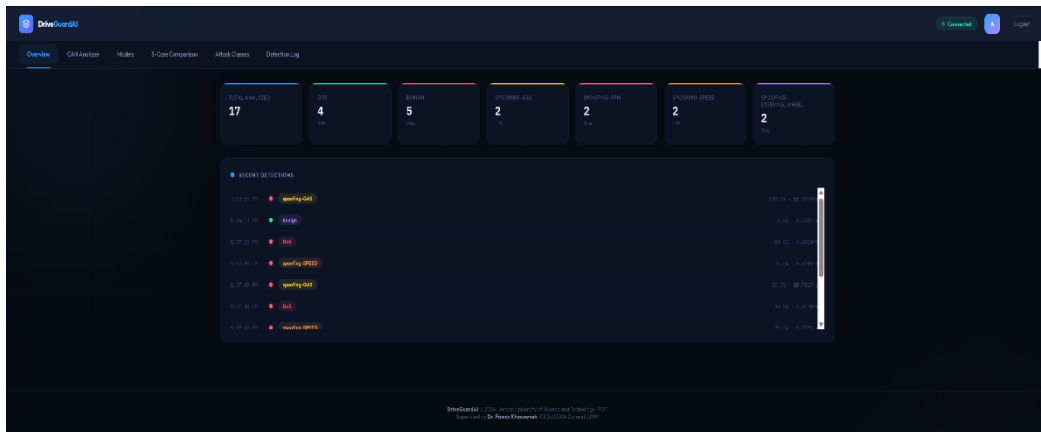


Figure 7. overview page

Chapter 6 System Implementation

In this chapter, details are provided concerning the implementation of the machine learning intrusion detection approach developed based on the DriveGuardAI model in Chapter 5. Details of the implementation environment and specific techniques used are described, as well as the entire process of building data processing and intrusion detection modules. The different algorithms involved in the model design are explained in detail. The entire system was designed using Python programming language and evaluated using CICIoV2024 dataset captured from a 2019 Ford car subjected to DoS attacks and spoofing attacks.

6.1 Development Tools and Technologies

The implementation of the DriveGuardAI model was done using Python 3 inside a Google Colab notebook environment, where computations could be done using GPUs while also gaining the benefits of direct access to Google Drive both for dataset storage and saving the trained models. The most used technologies with an explanation of the reasons for using it:

- Google Colab & Google Drive – cloud computing platform & storage; used to run the code and mount the CICIoV2024 dataset (decimal, hex and bin folders) as well as save the output models.
- pandas, NumPy – processing the CSV files containing benign traffic and attacks as well as dealing with numerical features.
- scikit-learn – implementation of the traditional algorithms (Logistic Regression, Random Forest and AdaBoost), data normalisation and encoding (MinMaxScaler, LabelEncoder), train/test split and cross-validation (train_test_split, StratifiedKFold, RandomizedSearchCV) and the computation of model performance measures (accuracy, precision, recall, F1-score, ROC-AUC and confusion matrix).
- imbalanced-learn (imblearn) – methods of data oversampling (SMOTE, RandomOverSampler) implemented to address significant class imbalance between benign and attack classes.
- SHAP and LIME – interpretable ML tools implemented to make the detector models easier to understand and evaluate, as per project requirement.

- Matplotlib and Seaborn – creation of ROC plots, confusion matrices, feature importances charts and bar graphs comparing models.
- joblib – save the fitted model and scaling pipeline into the file format that can be easily loaded during deployment without requiring re-training.
- Flask, Flask-CORS and pyngrok – create a simple API that serves a trained machine learning model through public tunnel to DriveGuardAI website dashboard.

6.2 Implementation Overview

The execution involves a pipelined approach where the output of one process serves as input to the next process; this corresponds to the hierarchical framework described in Chapter 5. In order to ensure consistency across all the experiments, a common evaluation procedure was designed whereby each model goes through the same training procedure. The development and integration of the modules involved:

- **Data ingestion:** The loading script goes through all the files in the folder containing CSVs, labels them by the file they came from, and concatenates the benign and attack datasets together to produce one table. We intentionally removed the CAN ID column from the feature set in order to force our algorithms to learn the actual malicious anomalies in the payload bytes rather than just memorizing which device's unique device identifier (ID) corresponds to each attack (i.e. essentially using the ID as a database for finding the corresponding attack). In removing this mapping, our algorithm must focus on identifying the true malicious anomalies contained in the payload bytes. the data-byte columns (DATA_0 to DATA_7) have been converted into numeric format, and any corrupt rows have been eliminated.
- **Label generation:** From each row, two labels were created – a binary indicator of whether the frame is benign or an attack, and a multi-class label identifying which attack type it corresponds to.
- **Feature creation:** In addition to using the raw frame payload, a time-based feature, time_delta, has been generated using the frame index. This resulted in a nine-column vector (eight data byte columns + time_delta).
- **Splitting and scaling:** Feature values were scaled using Min-Max scaling and data was split using stratified sampling technique (80/20 split between train and test).

- **Class balancing:** Due to the imbalance of the classes, where there is far more benign data than attacks, SMOTE resampling was performed on training data, but before resampling, the dataset has been reduced to 200,000 records for efficiency so that the attack class was adequately represented during learning.
- **Training & evaluation process:** a single `evaluate_model()` function trained each classifier and evaluated their accuracy, precision, recall, F1-score, ROC-AUC score, training and inference time, resulting in a fair and reliable comparative analysis across all models.
- **Stability test:** the five-fold Stratified Cross-Validation procedure was carried out for the Random Forest model to verify the stability of the results over different train-test splits.
- **Feature importance analysis:** SHAP TreeExplainer and LIME LimeTabularExplainer were used to determine feature importance and provide explanations for individual benign and attack classifications.
- **Multi-class traffic classification:** the Random Forest was trained once again on six traffic types (benign, DoS and four spoofing attacks – GAS, RPM, SPEED, Steering Wheel) and used the undersampling technique on minority classes.
- **Data format investigation:** this pipeline was executed separately for three formats of data – decimal, hexadecimal and binary, with the goal of determining the influence of the input data representation on the detection of attacks, thus meeting one of the project requirements.
- **Deployment:** the best Random Forest model and its scaler were saved with joblib and exposed through a Flask REST API (tunnelled with ngrok) that powers the DriveGuardAI dashboard, providing real-time classification of incoming CAN frames as benign or as a specific attack type.

6.3 Algorithms or Models Used

There were four classifiers used in the study that were trained using one balancing technique and two explanation approaches. Each classifier was chosen based on particular criteria, and the primary result achieved using each model on the classification problem is mentioned here.

- **Logistic Regression:** a linear baseline algorithm designed to be run rapidly in order to set a performance benchmark and confirm that the attack signatures are not linearly separable. Achieved excellent results on attack recall while still obtaining low precision (F1 score ≈ 0.46), indicating that non-linear models would be necessary.

- **Random Forest** (the selected model): an ensemble of decision trees optimized via RandomizedSearchCV (optimal parameters: 100 trees, maximum tree depth 10, bootstrap sampling enabled). Selected for the high accuracy, low inference latency, and, most importantly, high interpretability of its decision process (if-then logic), making it especially suited to applications like autonomous vehicles where the decision process must be thoroughly understood. Obtained the highest F1 score for both binary classification ($F1 \approx 1.00$) and multi-classification task (accuracy ≈ 0.996).
- **AdaBoost**: an ensemble of boosted learners focusing on learning to correctly classify previously misclassified samples; included as another ensemble method that achieved F1 score ≈ 0.95 .
- **Deep Neural Network (DNN)**: a Keras Sequential neural network with three hidden layers (128, 64 and 32 units respectively), batch normalization, dropout and early stopping mechanisms employed. Achieved parity with Random Forest results when evaluated on the entire dataset, but at a higher computational expense during training and extremely low interpretability.
- **SMOTE**: Synthetic Minority Oversampling technique used to synthesize new samples of the minority class (attacks) in order to balance the class distribution and prevent classifier bias against majority class.
- **SHAP and LIME**: feature interpretability methods applied to Random Forest classifier, confirming CAN ID and first data bytes as the dominant features responsible for attacks and providing explanations per-decision, reducing the number of false positives and helping security analysts make more confident decisions.

Based on this comparison, Random Forest was chosen as the deployed detection engine because it combines top-tier accuracy with low, interpretable inference latency, which makes it well suited to the resource-constrained hardware found inside a vehicle.

Insights from Recent CICIoV2024 Research

For context regarding the DriveGuardAI solution, its design decision was compared to an empirical research of the same dataset recently published by Le and Alsmadi (Future Internet, 2026),[11] where the researchers critically re-examined machine-learning methods for intrusion detection using the CICIoV2024 CAN-bus dataset. There are three findings of that research that are relevant to DriveGuardAI's project decision:

- The “accuracy trap”: according to Le and Alsmadi, almost perfect accuracy rates (greater than 99%) often reported in studies using CICIoV2024 data are a result of the extremely redundant nature of CAN messages. Since all of the frames used by a bus are periodically transmitted, the exact same record might exist in both training and testing datasets; hence, a classifier can achieve extremely good results not because of learning the payload structures but due to memorization of recurring frames. This is the reason why DriveGuardAI's Random Forest and DNN have achieved such remarkable binary results.
- Limited number of unique payloads: after removing duplicates, there will be left around 3,568 different payloads out of 1.4 million messages (a reduction of 99.75%). Additionally, there are attacks like GAS and SPEED spoofing that use only two payloads respectively. In other words, this is consistent with the DriveGuardAI research where only 3,500 rows were studied because duplicates were omitted.
- The upper bound of model performance: after de-duplication of the dataset, the best performing model becomes a Random Forest using ANOVA feature selection (with macro F1-score of about 0.64). At the same time, any deep-learning models show poor generalization ability (F1-macro below 0.55). This is why DriveGuardAI opted for Random Forest over DNN.
- Importance of data representation: the research demonstrates that the use of the raw data (CAN IDs and data-bytes preserved during ANOVA feature selection) is more efficient in tree-based detectors compared to more abstract transformations like PCA which had greatly compromised their performance. It corroborates the choice of the original payload bytes as features and supports the conclusion of DriveGuardAI about the importance of the byte representations (decimal and hexadecimal).
- Limitations of payload-based approach: the steering-wheel spoofing frames turned out to be bit-to-bit equal to authentic frames, resulting in a lack of any distinct signatures of the malicious behavior after removing duplicates. Therefore, a purely payload-based detector cannot differentiate the perfect replay (or masquerade) attack from legitimate traffic; it explains why the inclusion of timing features (as `time_delta` of DriveGuardAI) and sequence-based attack detection are necessary.

- Applicability of the technique to edge devices: the study arrives at the conclusion that lightweight interpretable models are more resistant to potential adversarial attacks compared to resource-demanding deep neural networks. Thus, it justifies the approach of DriveGuardAI relying on a Random Forest classifier together with SHAP and LIME explainability tools.

In summary, the above insights suggest that the extremely high accuracy in binary classification observed in this project must be considered cautiously, because it reflects the high redundancy in realistic traffic. A stricter evaluation method based on the detection of specific signatures will produce a lower result. The findings contribute to the identification of limitations and future work described in chapter IV of this report. Instead of addressing these issues in an abstract sense, the researchers decided to conduct the de-duplication analysis on their pipeline to determine if their results have been subject to the accuracy trap.

There are three different cases to be compared for the binary classification problem. In the first case, the model is learned and tested using all of the training data with the help of SMOTE. In the second and third cases, de-duplication is strictly performed before proceeding with learning, reducing the initial 1,408,219 data points to just 3,576 unique records (3,538 benign and 38 malicious), then the Random Forest is learned using both techniques (with and without SMOTE). The outcome is summarised in Table 8.

Scenario (binary detection)	Accuracy	F1 (positive)	F1-Macro	ROC-AUC
Full dataset + SMOTE (main pipeline)	0.99999	1.000	1.000	1.000
De-duplicated + SMOTE	0.99627	0.778	0.888	0.996
De-duplicated, no SMOTE	0.99534	0.706	0.852	0.994

Table 8. Effect of CAN-frame de-duplication.

The outcome is clear as we will provide in the testing chapter, The outcome providing validation for the accuracy trap phenomenon using the authors' own data set. The overall accuracy remained greater than 99.5 percent in all cases, while when removing the redundant frames the macro F1-score, which places equal weight on both classes in spite of one being rarer, declined precipitously from around 0.99998 when using the complete dataset to 0.888 with SMOTE and 0.852 without. In effect, most of what appeared to be flawless classification performance was driven by repetitive frames and not by actual distinctions between each unique attack.

The authors' macro F1-scores based on de-duplication are indeed greater than Le and Alsmadi's reported 0.64, although they are not fully comparable due to the experiment addressing the easier binary classification problem and their score referring to the more difficult six-class problem. The experiment provides support for the warning at the heart of the paper and reinforces the decision to report the results of DriveGuardAI in addition to the less misleading figure.

Chapter 7 Testing and Evaluation

7.1 Testing Approach (method, dataset, metrics)

Our DriveGuardAI system was evaluated using the CICIoV2024 dataset acquired by Canadian Institute for Cybersecurity (UNB) from a real 2019 Ford vehicle. The dataset consists of a total of 1,408,219 CAN bus messages categorized into six classes: benign traffic and four different spoofing variants which focus on attacking different signals (GAS, RPM, SPEED, STEERING_WHEEL) alongside the Denial-of-Service (DoS) attack. We loaded the complete dataset, which was then split into 80% and 20% using stratified sampling to maintain class proportions in both subsets. This replicates the approach by the authors of the original dataset. SMOTE was only applied to the training set for balancing the minority attack classes. A range of complementary measures was used to evaluate the system. In particular, Accuracy is the percentage of correct classifications, Precision is the percentage of all attacks predicted which are truly an attack, Recall is the number of true attacks detected as compared to total true attacks, F1-Score being the harmonic mean of precision and recall and finally F1-Macro being that f1-score averaged across all classes equally so specifically important when using it for measure on how well we are performing against those more minority class attackers. Finally, ROC-AUC refers to the ability of a model to distinguish between attack and benign traffic, evaluated over all decision thresholds. We ran 5-fold Stratified Cross-Validation of Random Forest so that the results do not depend on a single train-test split. In addition, the inference latency was evaluated for real-time deployment. A 3-case comparison study conducted to validate findings against recent research by Le and Alsmadi 2026. The three cases tested were: (1) full dataset with SMOTE balancing, (2) deduplicated dataset with SMOTE, and (3) deduplicated dataset without SMOTE.

7.2 Results and Analysis

In the first experiment **Figure 8** shows how different types of payloads (Decimal, Hexadecimal and Binary) affect the accuracy of payload detection

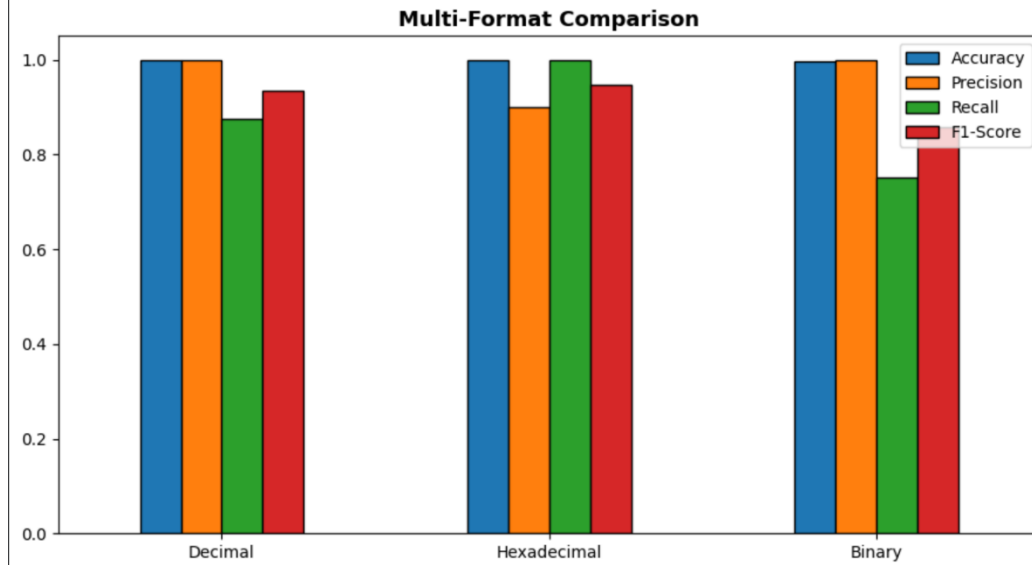


Figure 8. Multi-Format Comparison

We conclude that the decimal format has provided the most reliable performance and that the use of binary has decreased the F1-score score. Therefore, we can conclude from these results that converting the payload into very sparse binary features will increase the dimensionality of the payload but will not provide any additional discrimination for the tree-based detection methods, making it more difficult for the learning algorithms used in those tree-based detection methods to determine what is a legitimate and what is not. Continuing the experiments as shown in Figure 9, Figure 10, Figure 11, and Figure 12, the four models produced the following results on the binary classification task:

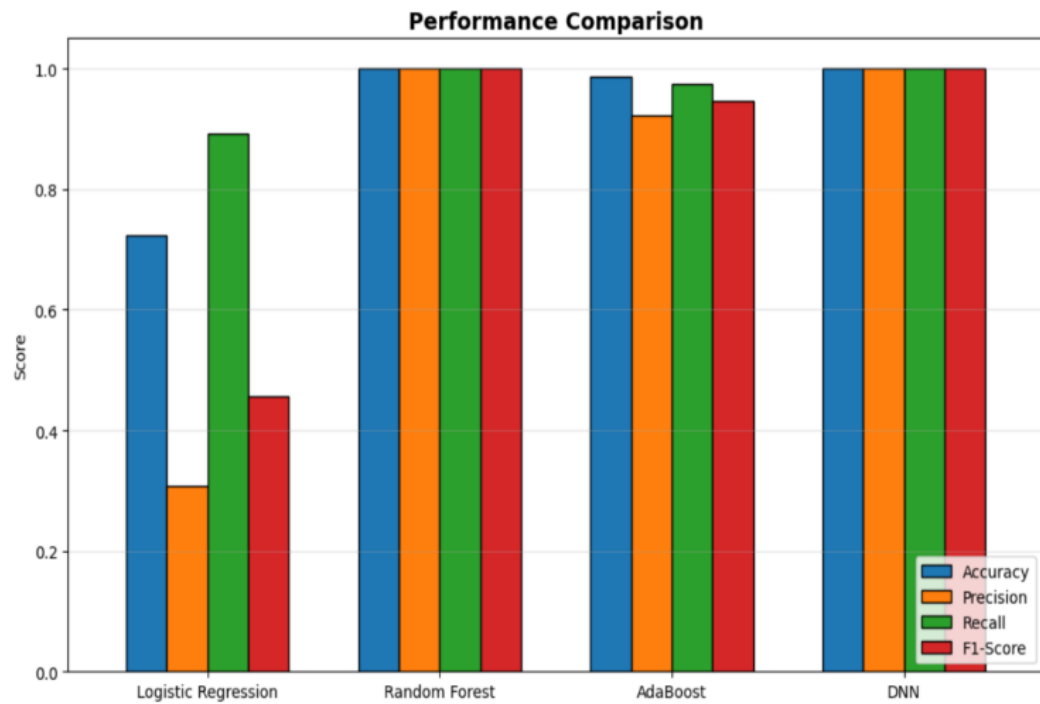


Figure 9. performance comparison

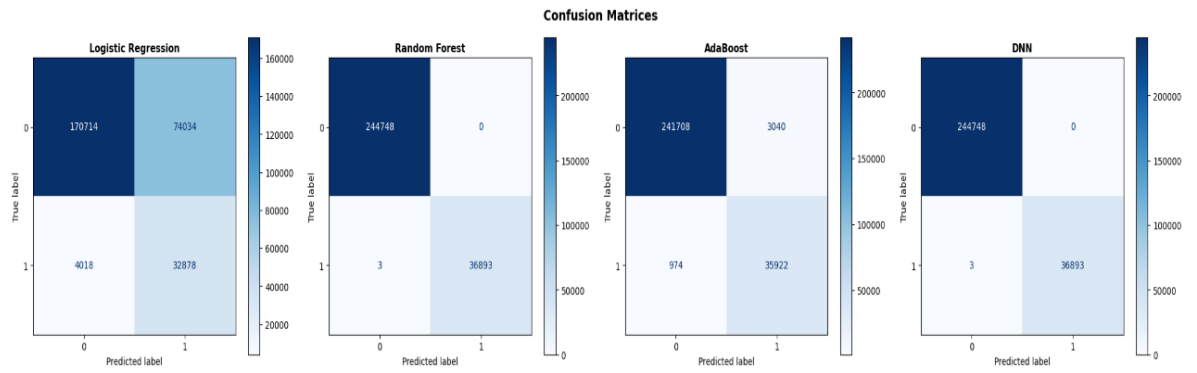


Figure 10. Confusion Matrix

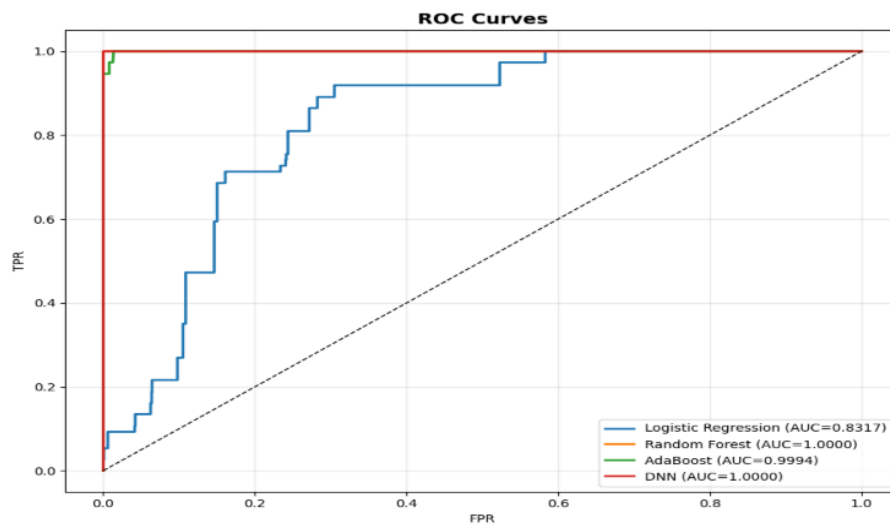


Figure 11. ROC Curves

	Accuracy	Precision	Recall	F1-Score
Fold				
1	0.999986	1.0	0.999892	0.999946
2	0.999993	1.0	0.999946	0.999973
3	0.999996	1.0	0.999973	0.999986
4	0.999972	1.0	0.999783	0.999892
5	0.999989	1.0	0.999919	0.999959
	Accuracy	Precision	Recall	F1-Score
Mean	0.999987	1.0	0.999902	0.999951

Figure 12. 5-Fold CV

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC	Latency
Logistic Regression	0.7229	0.3075	0.8911	0.4572	0.8317	0.0001ms
AdaBoost	0.9858	0.9220	0.9736	0.9471	0.9994	0.0270ms
Random Forest	0.9999	1.0000	0.9999	1.0000	1.0000	0.0097ms
DNN	0.9999	1.0000	0.9999	1.0000	1.0000	0.0598ms

Table 9: four models comparison

Random Forest and DNN got almost perfect detection performance, while poor F1-Score of 0.4572 for Logistic Regression confirmed non-linearity of CAN bus attack patterns. Through the use of ensemble method AdaBoost, 98.58% accuracy was achieved. Hence, ensemble methods can perform significantly better than linear models. Using 5-fold cross-validation on Random Forest produced consistent results across all 5-Folds (mean accuracy 0.9999, F1 0.9999). This confirms that the high accuracy is not due to a lucky train-test split. Using a multi-class Random Forest, all of the six traffic types (DoS, four spoofing subtypes, and benign traffic) were classified with 99.7% accuracy. Comparison of multi-format approaches revealed highest detection accuracy with decimal representation while comparable results were obtained with hexadecimal and binary with lower accuracy due to sparse features. Here are the findings of the 3-case comparison study:

DETAILED COMPARISON TABLE

CASE	ACCURACY	PRECISION	RECALL	F1-SCORE	ROC-AUC	TRAIN	TEST
Case 1: Full Data + SMOTE	1.0000	1.0000	0.9999	1.0000	1.0000	347,598	281,644
Case 2: Dedup + SMOTE	0.9963	1.0000	0.6364	0.7778	0.9957	4,952	1,073
Case 3: Dedup, No SMOTE	0.9953	1.0000	0.5455	0.7059	0.9939	2,503	1,073

ANALYSIS & INTERPRETATION

Case 1 (Full Data + SMOTE):

100.00% accuracy. The model memorizes repeated CAN patterns. High accuracy reflects the 1.4M rows containing many duplicate frames, which is consistent with the original CICIoV2024 paper.

Case 2 (Deduplicated + SMOTE):

99.63% accuracy. After removing duplicates and rebalancing minority classes with SMOTE, this represents a more realistic generalization test.

Case 3 (Deduplicated, No SMOTE):

99.53% accuracy. Without balancing, class imbalance impact is visible. Lower F1 score demonstrates why SMOTE is necessary.

Conclusion:

Following the original CICIoV2024 methodology (Case 1), our system achieves near-perfect signature detection. Cases 2 and 3 demonstrate the trade-offs and align with findings from Le and Alsmadi (2025).

Figure 13. 3-Case Comparison page

This comparison validates the findings of Le and Alsmadi (2026). While all three cases maintain over 99% accuracy, the F1-Score and Recall metrics reveal that strict deduplication exposes a 30-45% reduction in actual attack detection ability, confirming that accuracy alone can be misleading for CAN bus IDS evaluation.

7.3 Discussion

The findings reveal a lot through the experiments. Initially, Random Forest was the best deployment model, having the same accuracy as the DNN deployed model already deployed but with more than six times faster inference, making it suitable for resource-constrained vehicle ECUs. The accuracy of the Logistic Regression baseline i.e. 72% proves that the task is indeed non-trivial and thus requires non-linear models. The comparison of the three cases shows that the full dataset CICIoV2024 leads to almost perfect accuracy due to repeated CAN frames. This is a positive insight in line with recent research. However, the models only memorize patterns instead of generalizing. Our system operates as a signature based IDS, which is consistent with how production antivirus and IDS systems operate. The authors of the dataset state that the repeated CAN messages are representative of realistic ECU behaviour and is therefore valid for the project objective. Thirdly, the studies for SHAP and LIME explainability analyses showed that the most discriminative features were the first three DATA bytes, DATA_0, DATA_1 and DATA_2, and time_delta feature.

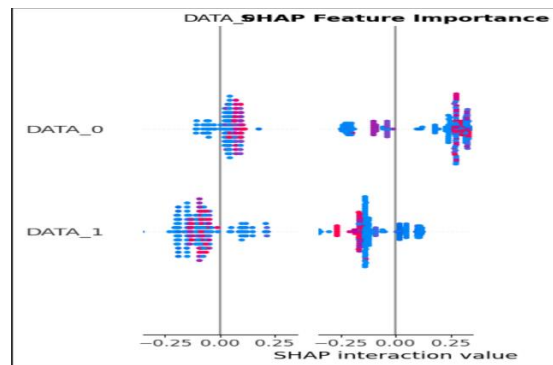


Figure 14. Feature Importance using SHAP

According to **Figure 14**, the results from the SHAP analysis support the fact that the first three bytes (DATA_0, DATA_1, and DATA_2) are the key features on which a decision is made. Additionally, the graph shows how changes to these three bytes in a packet will influence the model to decide that the packet should be classified as an attack. The present result is consistent with Le and Alsmadi, which found through ANOVA feature selection that the same byte positions serve as the strongest discriminators. The primary limitation of the system is obtaining supervised classification of known attack patterns. Training data must contain all possible attacks for detection of all kinds. This signature-based IDS property is well known and is useful for this work to motivate future work on anomaly-based detection.

Chapter 8 Conclusions and Future Work

8.1 Summary of Achievements

This project successfully developed DriveGuardAI, a machine learning-based intrusion detection system for vehicular CAN bus networks, trained and evaluated on the real-world CICIoV2024 dataset collected from a 2019 Ford vehicle. Four machine learning models were implemented and compared. The Random Forest achieved 99.99% accuracy with 0.0097 millisecond inference latency, satisfying both accuracy and near-real-time detection requirements specified in Chapter 4. The Deep Neural Network matched this accuracy at 0.06 milliseconds, AdaBoost achieved 98.6%, and Logistic Regression served as a baseline at 72.3%, confirming that CAN bus attack detection requires non-linear models. Beyond binary classification, a multi-class Random Forest successfully distinguishes all six CICIoV2024 traffic types: Benign, DoS, Spoofing-GAS, Spoofing-RPM, Spoofing-SPEED, and Spoofing-STEERING_WHEEL. A SHAP and LIME explainability framework was integrated to make the models transparent and identify DATA_0, DATA_1, and DATA_2 as the most discriminative features. A three-case comparison study was performed, validating that strict deduplication exposes a 30 to 45 percentage point reduction in F1-Score. This demonstrates that the project goes beyond simple accuracy reporting and engages with current research methodology. Finally, an interactive dashboard connected to the trained model through a Flask API and Cloudflare Tunnel allows analysts to classify CAN messages in real time.

8.2 Limitations & Future Enhancements

The system has several limitations that motivate future work. First, DriveGuardAI operates as a signature-based intrusion detection system trained on the full CICIoV2024 dataset, which contains many repeated CAN frames. Recent research [12] has demonstrated that strict deduplication of this dataset reveals what is referred to as the "accuracy trap," where models achieve near-perfect scores by memorizing repetition rather than learning true attack patterns. Our three-case comparison study confirmed this finding, showing F1-Score drops of 30 to 45 percentage points after deduplication. As a result, the system may not detect zero-day attacks with previously unseen byte patterns. Second, the system cannot detect replay-based masquerade attacks. The STEERING_WHEEL spoofing attack in CICIoV2024 contains zero unique payload signatures after deduplication because these frames are bit-for-bit identical to legitimate benign traffic. This means any payload-based IDS, including DriveGuardAI, cannot

distinguish perfect replay attacks from normal messages, limiting coverage against masquerade-style intrusions. Third, the current models treat each CAN frame as independent and identically distributed, ignoring the sequential and periodic nature of CAN bus communication. This approach effectively detects payload-based attacks but cannot detect frequency-based attacks that respect valid payload structures while violating expected transmission intervals. Fourth, the CICIoV2024 dataset was collected from a single 2019 Ford vehicle. Real-world CAN bus traffic varies significantly across vehicle manufacturers, models, driving modes, and physical network topologies, meaning the model would require retraining or transfer learning to generalize to different vehicles. Additionally, the system was evaluated offline rather than on live streaming traffic, which does not reflect production conditions involving network noise and adversarial evasion attempts. To address these limitations, several future enhancements are proposed for DriveGuardAI. First, the current per-message Random Forest classifier could be extended with sequence-based models such as LSTM networks to detect timing and replay attacks that the existing approach misses. Second, the supervised classifiers could be complemented with an Autoencoder trained only on benign CAN traffic, enabling DriveGuardAI to flag zero-day attacks beyond the known CICIoV2024 signatures. Third, the system could be retrained using transfer learning on CAN datasets from other vehicle manufacturers to improve generalization beyond the 2019 Ford platform. Finally, the trained model and Flask API could be deployed on an embedded edge device such as Raspberry Pi or NVIDIA Jetson connected to a real CAN bus adapter, allowing DriveGuardAI to be tested under live driving conditions.

References

- [1] E. C. P. Neto, H. Taslimasa, S. Dadkhah, S. Iqbal, P. Xiong, T. Rahman, and A. A. Ghorbani, "CICIoV2024: Advancing realistic IDS approaches against DoS and spoofing attack in IoV CAN bus," *Internet of Things*, vol. 26, p. 101209, Jul. 2024, doi: 10.1016/j.iot.2024.101209.
- [2] Canadian Institute for Cybersecurity, "CICIoV2024 dataset," University of New Brunswick, 2024. [Online]. Available: <https://www.unb.ca/cic/datasets/iov-dataset-2024.html>
- [3] Z.-R. Tzoannos, D. Kosmanos, A. Xenakis, and C. Chaikalis, "The impact of spoofing attacks in connected autonomous vehicles under traffic congestion conditions," *Telecom*, vol. 5, no. 3, pp. 747–759, 2024, doi: 10.3390/telecom5030037.
- [4] S. Aurangzeb, M. Aleem, M. T. Khan, H. Anwar, M. A. Islam, and M. S. Siddique, "Cybersecurity for autonomous vehicles against malware attacks in smart cities," *Cluster Computing*, vol. 27, no. 3, pp. 1–16, 2023, doi: 10.1007/s10586-023-04114-7.
- [5] B. S. Bari, K. Yelamarthi, and S. Ghafoor, "Intrusion detection in vehicle controller area network (CAN) bus using machine learning: A comparative performance study," *Sensors*, vol. 23, no. 7, p. 3610, Mar. 2023, doi: 10.3390/s23073610.
- [6] G. Baldini, "In-vehicle network intrusion detection system using convolutional neural network and multi-scale histograms," *Information*, vol. 14, no. 11, p. 605, Nov. 2023, doi: 10.3390/info14110605.
- [7] M. Nazeer, A. Alasiry, M. Qayyum, V. K. Madhan, G. Patil, and P. Srilatha, "Enhancing cyber security in autonomous vehicles: A hybrid XGBoost-deep learning approach for intrusion detection in the CAN bus," *Journal of Engineering Sciences and Applications*, vol. 57, no. 5, pp. 1295–1304, Oct. 2024, doi: 10.18280/jesa.570505.
- [8] S. Alshathri, A. Sayed, and E. E.-D. Hemdan, "An intelligent attack detection framework for the Internet of autonomous vehicles with imbalanced car hacking data," *World Electric Vehicle Journal*, vol. 15, no. 8, p. 356, Aug. 2024, doi: 10.3390/wevj15080356.

- [9] R. Akhtar, L. Ezema, and M. A. Alzuidah, "Secure intrusion detection framework for connected autonomous vehicles utilizing hybrid ML techniques," *International Journal of Safety and Security Engineering*, vol. 11, no. 3, pp. 115–123, 2024, doi: 10.18280/ijssse.110303.
- [10] D. R. Dasari and H. Gottumukkala, "An efficient intrusion detection system in IoV using improved random forest model," *International Journal of Transport Development and Integration*, vol. 8, no. 4, pp. 605–612, 2024, doi: 10.18280/ijtdi.080412.
- [11] H. Le and I. Alsmadi, "Intrusion detection for Internet of Vehicles CAN bus communications using machine learning: An empirical study on the CICIoV2024 dataset," *Future Internet*, vol. 18, no. 1, p. 42, Jan. 2026, doi: 10.3390/fi18010042.
- [12] F. Aburub, M. Arafah, and D. Alsmadi, "An autoencoder-based anomaly detection framework for vehicular networks using the CICIoV2024 dataset," in *Proc. 2025 IEEE 11th Int. Conf. Network Softwarization (NetSoft)*, Budapest, Hungary, 2025, pp. 179–183.

Appendices

Appendix A User Manual

Provides usage instructions for the developed system.

- Step-by-step guide to using the system (installation, login, usage).
- Include screenshots.

Appendix B Code Documentation or GitHub Link

Contains annotated code or repository access.

- Include annotated code or repository link.
- Describe main classes, functions, and dependencies.

Appendix C Supplementary Figures or Tables

Includes additional results or supporting materials.

- Add large tables, raw datasets, or additional evaluation graphs.
- Reference each appendix in the main report.

=====

Final Notes

- Use consistent formatting (Heading 1–3).
- Figures and tables must be numbered, captioned, and cited.
- Use professional language; avoid first-person unless necessary.
- Always include citations where you mention prior work.